

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт

з лабораторної роботи № 2 з дисципліни
«Проектування алгоритмів»

«Неінформативний, інформативний та локальний пошук»

Виконав(ла)

ІП-341 Черепов Олександр Павлович
(шифр, прізвище, ім'я, по батькові)

Перевірив

Головченко М.М.
(прізвище, ім'я, по батькові)

Київ 2025

ЗМІСТ

1	МЕТА ЛАБОРАТОРНОЇ РОБОТИ	3
2	ЗАВДАННЯ	4
3	ВИКОНАННЯ.....	8
3.1	ПСЕВДОКОД АЛГОРИТМІВ.....	8
3.2	ПРОГРАМНА РЕАЛІЗАЦІЯ.....	11
3.2.1	<i>Вихідний код.....</i>	<i>11</i>
3.2.2	<i>Приклади роботи</i>	<i>15</i>
3.3	ДОСЛІДЖЕННЯ АЛГОРИТМІВ.....	16
	ВИСНОВОК	18
	КРИТЕРІЇ ОЦІНЮВАННЯ	19

1 МЕТА ЛАБОРАТОРНОЇ РОБОТИ

Мета роботи – розглянути та дослідити алгоритми неінформативного, інформативного та локального пошуку. Провести порівняльний аналіз ефективності використання алгоритмів.

2 ЗАВДАННЯ

Записати алгоритм розв’язання задачі у вигляді псевдокоду, відповідно до варіанту (таблиця 2.1).

Реалізувати програму, яка розв’язує поставлену задачу згідно варіанту (таблиця 2.1) за допомогою алгоритму неінформативного пошуку **АНП**, алгоритму інформативного пошуку **АП**, що використовує задану евристичну функцію *Func*, або алгоритму локального пошуку **АЛП** та **бектрекінгу**, що використовує задану евристичну функцію *Func*. Розробити власну евристичну функцію та порівняти її із заданою за варіантом.

Програму реалізувати на довільній мові програмування.

Увага! Алгоритм неінформативного пошуку **АНП**, реалізовується за принципом «AS IS», тобто так, як є, без додаткових модифікацій (таких як перевірка циклів, наприклад).

Провести серію експериментів для вивчення ефективності роботи алгоритмів. Кожний експеримент повинен відрізнятися початковим станом. Серія повинна містити не менше 20 експериментів для кожного алгоритму. Початковий стан зафіксувати у таблиці експериментів. За проведеними серіями необхідно визначити:

- середню кількість етапів (кроків), які знадобилось для досягнення розв’язку (ітерації);
- середню кількість випадків, коли алгоритм потрапляв в глухий кут (не міг знайти оптимальний розв’язок) – якщо таке можливе;
- середню кількість згенерованих станів під час пошуку;
- середню кількість станів, що зберігаються в пам’яті під час роботи програми.

Передбачити можливість обмеження виконання програми за часом (30 хвилин) та використання пам’яті (1 Гб).

Використані позначення:

– **8-ферзів** – Задача про вісім ферзів полягає в такому розміщенні восьми ферзів на шахівниці, що жодна з них не ставить під удар один одного. Тобто, вони не повинні стояти в одній вертикалі, горизонталі чи діагоналі.

– **8-puzzle** – гра, що складається з 8 однакових квадратних пластинок з нанесеними числами від 1 до 8. Пластинки поміщаються в квадратну коробку, довжина сторони якої в три рази більша довжини сторони пластинок, відповідно в коробці залишається незаповненим одне квадратне поле. Мета гри – переміщаючи пластинки по коробці досягти впорядкування їх по номерах, бажано зробивши якомога менше переміщень.

– **Лабіринт** – задача пошуку шляху у довільному лабіринті від початкової точки до кінцевої з можливими випадками відсутності шляху. Структура лабіринту зчитується з файлу, або генерується програмою.

– **COLOR** – Задача розфарбування карти самостійно обраної країни, не менше 20 регіонів (областей). Необхідно розфарбувати карту не більше ніж у 4 різні кольори. Мається на увазі приписування кожному регіону власного кольору так, щоб кольори сусідніх регіонів відрізнялись. Використовувати евристичну функцію, яка повертає кількість пар суміжних вузлів, що мають однаковий колір (тобто кількість конфліктів).

– **TSP** – Задача комівояжера. Необхідно обійти усі вершини графа на 100 вершин і повернутись у початкову. Вагу ребер задати випадково у діапазоні від 10 до 30. Довжина знайденого шляху повинна бути не більше 2000.

– **BCTR** – алгоритм пошуку із поверненнями (backtracking).

– **LDFS** – Пошук вглиб з обмеженням глибини.

– **BFS** – Пошук вшир.

– **IDS** – Пошук вглиб з ітеративним заглибленням.

– **A*** – Пошук A*.

– **RBFS** – Рекурсивний пошук за першим найкращим співпадінням.

– **HILL** – Пошук зі сходженням на вершину з використанням із використанням руху вбік (на 100 кроків) та випадковим перезапуском (кількість необхідних разів запуску визначити самостійно).

- **ANNEAL** – Локальний пошук із симуляцією відпалу. Робоча характеристика – залежність температури T від часу роботи алгоритму t . Можна розглядати лінійну залежність: $T = 1000 - k \cdot t$, де k – змінний коефіцієнт.
- **BEAM** – Локальний променевий пошук. Робоча характеристика – кількість променів k . Експерименти проводили із кількістю променів від 2 до 21.
- **F1** – кількість пар ферзів, які б'ють один одного з урахуванням видимості (ферзь A може стояти на одній лінії з ферзем B , проте між ними стоїть ферзь C ; тому A не б'є B).
- **F2** – кількість пар ферзів, які б'ють один одного без урахування видимості.
- **F3** – кількість ферзів, які стоять не на своїй місцях, в порівнянні з одним з еталонних розв'язків.
- **H1** – кількість фішок, які не стоять на своїх місцях.
- **H2** – Манхетенська відстань.
- **H3** – Евклідова відстань.
- **MRV** – Евристика мінімальної кількості значень.
- **DGR** – Ступенева евристика.
- **N1** – Евристика найближчого сусіда.

Таблиця 2.1 – Варіанти алгоритмів

№	Задача	АНП	АП	АЛП	Func
1	COLOR	BCTR		HILL	MRV
2	COLOR	BCTR		ANNEAL	MRV
3	COLOR	BCTR		BEAM	MRV
4	COLOR	BCTR		HILL	DGR
5	COLOR	BCTR		ANNEAL	DGR
6	COLOR	BCTR		BEAM	DGR
7	8-ферзів		A*	HILL	F1
8	8-ферзів		A*	ANNEAL	F2
9	8-ферзів		RBFS	BEAM	F3

10	8-ферзів		RBFS	HILL	F1
11	8-ферзів		A*	ANNEAL	F2
12	8-ферзів		A*	BEAM	F3
13	8-ферзів	BFS	RBFS		F1
14	8-ферзів	BFS	RBFS		F2
15	8-ферзів	IDS	A*		F3
16	8-ферзів	IDS	A*		F1
17	8-ферзів	IDS	RBFS		F2
18	Лабіринт	LDFS	A*		H3
19	8-puzzle	LDFS	A*		H1
20	8-puzzle	BFS	A*		H2
21	TSP	BCTR		HILL	N1
22	TSP	BCTR		ANNEAL	N1
23	TSP	BCTR		BEAM	N1
24	TSP	BCTR		HILL	N1
25	TSP	BCTR		ANNEAL	N1
26	TSP	BCTR		BEAM	N1
27	Лабіринт	BFS	A*		H3
28	8-puzzle	BFS	A*		H2
29	8-puzzle	IDS	RBFS		H1
30	8-puzzle	IDS	RBFS		H2
31	Лабіринт	LDFS	A*		H2
32	Лабіринт	LDFS	RBFS		H3
33	Лабіринт	BFS	A*		H2
34	Лабіринт	BFS	RBFS		H3
35	Лабіринт	IDS	A*		H2
36	Лабіринт	IDS	RBFS		H3

3 ВИКОНАННЯ

3.1 Псевдокод алгоритмів

Алгоритм неінформативного пошуку з поверненням (BCTR):

Головна процедура:

```
Const: SIZE = 100
Const: WEIGHT_RANGE = [10, 30]
Const: MAX_COST = 2000

procedure SolveTspBctr()
    global stats
    stats = initialize map with keys "status", "processing_time",
    "iterations", "dead_ends", "generated_nodes", "max_memory"

    visited = array of size SIZE initialized to False
    visited[0] = True

    matrix = GenerateWeightedMatrix(SIZE, WEIGHT_RANGE)

    start_time = GetCurrentTime()
    found = Backtrack(matrix, 0, 0, 1, visited)
    stats["processing_time"] = GetCurrentTime() - start_time

    if found is True then
        stats["status"] = "Found"
    else
        stats["status"] = "Fail"
    end if

    return stats
end procedure
```

Алгоритм пошуку:

```
procedure Backtrack(matrix, u, currentCost, count, visited)
    global stats
    stats["iterations"] = stats["iterations"] + 1
    if count > stats["max_memory"] then
        stats["max_memory"] = count
    end if

    if count == SIZE then
        if matrix[u][0] exists then
            totalCost = currentCost + matrix[u][0]
            if totalCost <= MAX_COST then
                return True
            end if
        end if

        stats["dead_ends"] = stats["dead_ends"] + 1
        return False
    end if
```



```

    for v from 0 to SIZE - 1 do
        if visited[v] is False then
            stats["generated_nodes"] = stats["generated_nodes"] + 1
            newCost = currentCost + matrix[u][v]

            if newCost <= MAX_COST then
                visited[v] = True

                if Backtrack(matrix, v, newCost, count + 1, visited) is
True then
                    return True
                end if

                visited[v] = False
            else
                stats["dead_ends"] = stats["dead_ends"] + 1
            end if
        end if
    end for

    stats["dead_ends"] = stats["dead_ends"] + 1
    return False
end procedure

```

Алгоритм локального пошуку з симуляцією відпалу (ANNEAL, N1):

Головна процедура:

```

Const: SIZE = 100
Const: WEIGHT_RANGE = [10, 30]
Const: MAX_COST = 2000
Const: INITIAL_TEMP = 1000
Const: MIN_TEMP = 0.001
Const: MAX_ITERATIONS = 100000

procedure SolveTspAnneal()
    global stats
    stats = initialize map with keys "status", "processing_time",
"iterations", "dead_ends", "generated_nodes", "max_memory"

    matrix = GenerateWeightedMatrix(SIZE, WEIGHT_RANGE)

    start_time = GetCurrentTime()

    currentPath = InitPath(matrix)
    currentCost = CalculateCost(matrix, currentPath)

    stats["max_memory"] = SIZE
    bestCost = currentCost

    k = INITIAL_TEMP / MAX_ITERATIONS
    t = 0
    currentTemp = INITIAL_TEMP

    while currentTemp >= MIN_TEMP and t < MAX_ITERATIONS do

```

```

t = t + 1
stats["iterations"] = stats["iterations"] + 1
stats["generated_nodes"] = stats["generated_nodes"] + 1

newPath = Copy(currentPath)
i = Random(0, SIZE - 1)
j = Random(0, SIZE - 1)
Ensure i < j

Reverse segment in newPath from i to j

newCost = CalculateCost(matrix, newPath)
delta = newCost - currentCost
accepted = False

if delta < 0 then
    accepted = True
else
    probability = exp(-delta / currentTemp)
    if Random(0, 1) < probability then
        accepted = True
    end if
end if

if accepted is True then
    currentPath = newPath
    currentCost = newCost

    if currentCost < bestCost then
        bestCost = currentCost
    end if
else
    stats["dead_ends"] = stats["dead_ends"] + 1
end if

currentTemp = INITIAL_TEMP - (k * t)
end while

stats["processing_time"] = GetCurrentTime() - start_time

if bestCost <= MAX_COST then
    stats["status"] = "Found"
else
    stats["status"] = "Fail"
end if

return stats
end procedure

```

Допоміжна процедура ініціалізації (N1):

```

procedure InitPath(matrix)
    unvisited = Set of all cities {0..SIZE-1}
    Remove 0 from unvisited

```

```

    currentCity = 0
    path = [0]

    while unvisited is not empty do
        nextCity = Find city in unvisited with minimum distance to
currentCity

        Append nextCity to path
        Remove nextCity from unvisited
        currentCity = nextCity
    end while

    return path
end procedure

```

3.2 Програмна реалізація

3.2.1 Вихідний код

Алгоритм неінформативного пошуку із поверненням (BCTR):

```

import time
from generate import generateWeightedMatrix

SIZE = 100
WEIGHT_RANGE = [10, 30]
MAX_COST = 2000

def solve_tsp_bctr():
    global stats
    stats = {"status": None, "processing_time": 0, "iterations": 0,
"dead_ends": 0, "generated_nodes": 0, "max_memory": 0}

    visited = [False]*SIZE
    visited[0] = True
    matrix = generateWeightedMatrix(SIZE, WEIGHT_RANGE)

    start_time = time.time()
    found = backtrack(matrix, 0, 0, 1, visited)

    stats["processing_time"] = time.time() - start_time
    stats["status"] = "Found" if found else "Fail"

    return stats

def backtrack(matrix, u, current_cost, count, visited):
    global stats
    stats["iterations"] += 1

```

```

if count > stats["max_memory"]:
    stats["max_memory"] = count

if count == SIZE:
    if matrix[u][0]:
        total_cost = current_cost + matrix[u][0]
        if total_cost <= MAX_COST:
            return True

    stats["dead_ends"] += 1
    return False

for v in range(SIZE):
    if not visited[v]:
        stats["generated_nodes"] += 1
        new_cost = current_cost + matrix[u][v]

        if new_cost <= MAX_COST:
            visited[v] = True

            if backtrack(matrix, v, new_cost, count + 1, visited):
                return True

            visited[v] = False
        else:
            stats["dead_ends"] += 1

stats["dead_ends"] += 1
return False

```

Алгоритм локального пошуку з симуляцією відпалу (ANNEAL, N1):

```

import time
import math
from random import randint, random
from generate import generateWeightedMatrix

SIZE = 100
WEIGHT_RANGE = [10, 30]
MAX_COST = 2000
INITIAL_TEMP = 1000
MIN_TEMP = 0.001
MAX_ITERATIONS = 100000

def solve_tsp_anneal():
    global stats
    stats = {

```

```

        "status": None,
        "processing_time": 0,
        "iterations": 0,
        "dead_ends": 0,
        "generated_nodes": 0,
        "max_memory": 0
    }

    matrix = generateWeightedMatrix(SIZE, WEIGHT_RANGE)

    start_time = time.time()

    current_path = init_path(matrix)
    current_cost = calculate_cost(matrix, current_path)

    stats["max_memory"] = SIZE

    best_cost = current_cost

    k = INITIAL_TEMP / MAX_ITERATIONS
    t = 0
    current_temp = INITIAL_TEMP

    while current_temp >= MIN_TEMP and t < MAX_ITERATIONS:
        t += 1
        stats["iterations"] += 1
        stats["generated_nodes"] += 1

        new_path = list(current_path)

        i = randint(0, SIZE-1)
        j = randint(0, SIZE-1)
        while i == j:
            j = randint(0, SIZE-1)

        i, j = sorted([i, j])
        new_path[i:j+1] = reversed(new_path[i:j+1])

        new_cost = calculate_cost(matrix, new_path)
        delta = new_cost - current_cost

        accepted = False

        if delta < 0:
            accepted = True
        else:
            prob = math.exp(-delta / current_temp)
            if random() < prob:
                accepted = True

```

```

        if accepted:
            current_path = new_path
            current_cost = new_cost

            if current_cost < best_cost:
                best_cost = current_cost
        else:
            stats["dead_ends"] += 1

        current_temp = INITIAL_TEMP - (k * t)

    stats["processing_time"] = time.time() - start_time
    stats["status"] = "Found" if best_cost <= MAX_COST else "Fail"

    return stats

def calculate_cost(matrix, path):
    cost = 0
    for i in range(len(matrix) - 1):
        cost += matrix[path[i]][path[i+1]]

    cost += matrix[path[-1]][path[0]]
    return cost

def init_path(matrix, start=0):
    unvisited = set(range(len(matrix)))
    unvisited.remove(start)

    cur = start
    path = [start]

    while unvisited:
        next_node = min(unvisited, key=lambda node: matrix[cur][node])

        path.append(next_node)
        unvisited.remove(next_node)
        cur = next_node

    return path

```

Генерація матриці зваженого графа (generate.py):

```

from random import randint

def generateWeightedMatrix(size, weight_range):
    output_matrix = [[0]*size for _ in range(size)]
    for i in range(size):
        for j in range(i+1, size):

```

```

        value = randint(weight_range[0], weight_range[1])
        output_matrix[i][j] = value
        output_matrix[j][i] = value

    return output_matrix

```

3.2.2 Приклади роботи

На рисунках 3.1 і 3.2 показані приклади роботи програми для різних алгоритмів пошуку.

```

PS C:\archive\EducationalWork\KPI\IA\lab2> python .\main_bctr.py
1. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 100, 'dead_ends': 0, 'generated_nodes': 99, 'max_memory': 100}
2. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 123, 'dead_ends': 23, 'generated_nodes': 122, 'max_memory': 100}
3. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 174, 'dead_ends': 119, 'generated_nodes': 218, 'max_memory': 100}
4. Stats:
{'status': 'Found', 'processing_time': 0.0009999275207519531, 'iterations': 100, 'dead_ends': 0, 'generated_nodes': 99, 'max_memory': 100}
5. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 100, 'dead_ends': 0, 'generated_nodes': 99, 'max_memory': 100}
6. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 100, 'dead_ends': 0, 'generated_nodes': 99, 'max_memory': 100}
7. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 100, 'dead_ends': 0, 'generated_nodes': 99, 'max_memory': 100}
8. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 100, 'dead_ends': 0, 'generated_nodes': 99, 'max_memory': 100}
9. Stats:
{'status': 'Found', 'processing_time': 18.403843879699707, 'iterations': 6630260, 'dead_ends': 24216140, 'generated_nodes': 24216239, 'max_memory': 100}
10. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 100, 'dead_ends': 0, 'generated_nodes': 99, 'max_memory': 100}
11. Stats:
{'status': 'Found', 'processing_time': 0.0, 'iterations': 100, 'dead_ends': 0, 'generated_nodes': 99, 'max_memory': 100}

```

Рисунок 3.1 – Алгоритм неінформативного пошуку із поверненням

```

PS C:\archive\EducationalWork\KPI\IA\lab2> python .\main_anneal.py
1. Stats:
{'status': 'Found', 'processing_time': 0.699500560760498, 'iterations': 100000, 'dead_ends': 2895, 'generated_nodes': 100000, 'max_memory': 100}
2. Stats:
{'status': 'Found', 'processing_time': 0.7076914310455322, 'iterations': 100000, 'dead_ends': 2822, 'generated_nodes': 100000, 'max_memory': 100}
3. Stats:
{'status': 'Found', 'processing_time': 0.6930627822875977, 'iterations': 100000, 'dead_ends': 2738, 'generated_nodes': 100000, 'max_memory': 100}
4. Stats:
{'status': 'Found', 'processing_time': 0.6887054443359375, 'iterations': 100000, 'dead_ends': 2897, 'generated_nodes': 100000, 'max_memory': 100}
5. Stats:
{'status': 'Found', 'processing_time': 0.7171792984008789, 'iterations': 100000, 'dead_ends': 2805, 'generated_nodes': 100000, 'max_memory': 100}
6. Stats:
{'status': 'Found', 'processing_time': 0.8596282005310059, 'iterations': 100000, 'dead_ends': 2912, 'generated_nodes': 100000, 'max_memory': 100}
7. Stats:
{'status': 'Found', 'processing_time': 0.8226006845855713, 'iterations': 100000, 'dead_ends': 2865, 'generated_nodes': 100000, 'max_memory': 100}
8. Stats:
{'status': 'Found', 'processing_time': 0.7959678173065186, 'iterations': 100000, 'dead_ends': 2906, 'generated_nodes': 100000, 'max_memory': 100}
9. Stats:
{'status': 'Found', 'processing_time': 0.791229248046875, 'iterations': 100000, 'dead_ends': 2903, 'generated_nodes': 100000, 'max_memory': 100}
10. Stats:
{'status': 'Found', 'processing_time': 0.7916910648345947, 'iterations': 100000, 'dead_ends': 2961, 'generated_nodes': 100000, 'max_memory': 100}
11. Stats:
{'status': 'Found', 'processing_time': 0.7931642532348633, 'iterations': 100000, 'dead_ends': 2907, 'generated_nodes': 100000, 'max_memory': 100}

```

Рисунок 3.2 – Алгоритм локального пошуку з симуляцією відпалу

3.3 Дослідження алгоритмів

В таблиці 3.1 наведені характеристики оцінювання алгоритму пошуку із поверненнями, задачі комівояжера для 20 початкових станів.

Таблиця 3.1 – Характеристики оцінювання алгоритму неінформативного пошуку із поверненнями

Початкові стани	Ітерації	К-сть гл. кутів	Всього вузлів	Всього вузлів у пам'яті
Стан 1	1980	3657	375	100
Стан 2	100	0	99	100
Стан 3	328	334	433	100
Стан 4	178299	4547375	4547474	100
Стан 5	12160	23067	23166	100
Стан 6	112	12	111	100
Стан 7	100	0	99	100
Стан 8	100	0	99	100
Стан 9	100	0	99	100
Стан 10	100	0	99	100
Стан 11	100	0	99	100
Стан 12	100	0	99	100
Стан 13	100	0	99	100
Стан 14	100	0	99	100
Стан 15	100	0	99	100
Стан 16	113	18	117	100
Стан 17	100	0	99	100
Стан 18	126	32	131	100
Стан 19	40149	85497	5596	100
Стан 20	100	0	99	100

В таблиці 3.2 наведені характеристики оцінювання алгоритму локального пошуку із симуляцією відпалу, задачі комівояжера для 20 початкових станів.

Таблиця 3.2 – Характеристики оцінювання алгоритму локального пошуку із симуляцією відпалу

Початкові стани	Ітерації	К-сть гл. кутів	Всього вузлів	Всього вузлів у пам'яті
Стан 1	100000	2891	100000	100
Стан 2	100000	2935	100000	100
Стан 3	100000	2937	100000	100
Стан 4	100000	2962	100000	100
Стан 5	100000	2949	100000	100
Стан 6	100000	2981	100000	100
Стан 7	100000	2885	100000	100
Стан 8	100000	2837	100000	100
Стан 9	100000	2936	100000	100
Стан 10	100000	2918	100000	100
Стан 11	100000	2831	100000	100
Стан 12	100000	2865	100000	100
Стан 13	100000	2831	100000	100
Стан 14	100000	2844	100000	100
Стан 15	100000	2902	100000	100
Стан 16	100000	2935	100000	100
Стан 17	100000	2914	100000	100
Стан 18	100000	2970	100000	100
Стан 19	100000	2850	100000	100
Стан 20	100000	2875	100000	100

ВИСНОВОК

При виконанні даної лабораторної роботи було розглянуто та досліджено алгоритми вирішення задачі комівояжера (TSP) для графа зі 100 вершин: алгоритм неінформативного пошуку із поверненнями (BCTR) та алгоритм локального пошуку з імітацією відпалу (ANNEAL) з використанням евристики найближчого сусіда (N1).

Проведений порівняльний аналіз показав, що алгоритм BCTR демонструє значну нестабільність результатів. Кількість ітерацій сильно залежала від випадковості початкового шляху і варіювалася від сотень до сотень тисяч, що підтверджує неефективність «сліпого» перебору для задач великої розмірності.

Натомість алгоритм ANNEAL виявився набагато ефективнішим. Поєднання евристики N1 для вдалого старту та механізму імітації відпалу для уникнення локальних мінімумів дозволило стабільно знаходити оптимальний маршрут за фіксовану кількість ітерацій (100 000).

КРИТЕРІЇ ОЦІНЮВАННЯ

Максимальний бал за лабораторну роботу №2 дорівнює – 10.

Критерії оцінювання:

- проектування алгоритму – 1;
- програмна реалізація алгоритму – 2;
- захист – 4;
- дослідження алгоритму – 2;
- звіт – 1.